This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Rethinking SNN Online Training and Deployment: Gradient-Coherent Learning via Hybrid-Driven LIF Model

Zecheng Hao[1,2,3] Yifan Huang[1,3] Zijie Xu[1,3,4] Wenxuan Liu[1,2*] Yuanhong Tang[1]
Zhaofei Yu[1,3,4] Tiejun Huang[1,2,4]
[1]School of Computer Science, Peking University
[2]State Key Laboratory for Multimedia Information Processing, Peking University
[3]Beijing Key Laboratory of Brain-inspired Spiking Large Models, Peking University
[4]Institute for Artificial Intelligence, Peking University
{haozecheng,yfhuang,liuwx66,ydtang,yuzf12,tjhuang}@pku.edu.cn,zjxu25@stu.pku.edu.cn

Abstract

Spiking Neural Networks (SNNs) are considered to have enormous potential in the future development of Artificial Intelligence due to their brain-inspired and energy-efficient properties. Compared to vanilla Spatial-Temporal Back-propagation (STBP) training methods, online training can effectively avoid the risk of GPU memory explosion. However, current online learning frameworks cannot tackle the gradient discrepancy problem between the forward and backward process, merely aiming to optimize the GPU memory, resulting in no performance advantages compared to the STBP-based models in the inference stage. To address the aforementioned challenges, we propose Hybrid-Driven Leaky Integrate-and-Fire (HD-LIF) model family for efficient online learning, which respectively adopt different spiking calculation mechanism in the upper-region and lower-region of the firing threshold. We theoretically point out that our learning framework can effectively separate temporal gradients and address the misalignment problem of surrogate gradients, as well as achieving full-stage optimization towards learning precision, memory complexity and power consumption. Experimental results have demonstrated that our scheme is enable to achieve state-of-the-art performance for multiple evaluation metrics, breaking through the traditional paradigm of SNN online training and deployment. Code is available at https://github.com/hzc1208/HD_LIF.

1. Introduction

Spiking Neural Networks (SNNs), as the third-generation neural network towards brain-inspired intelligence [29], have gained widespread attention from researchers in Artificial Intelligence community. SNNs utilize spiking neurons as the basic computing unit to transmit discrete spike firing sequences to the post-synaptic layer. Due to the fact that spiking neurons only emit spikes when the membrane potential exceeds the firing threshold, compared to the activation values in traditional Artificial Neural Networks (ANNs), spike sequences have sparse and event-driven properties, which can demonstrate superior computational efficiency and robustness on neuromorphic hardware [2, 5, 31, 33].

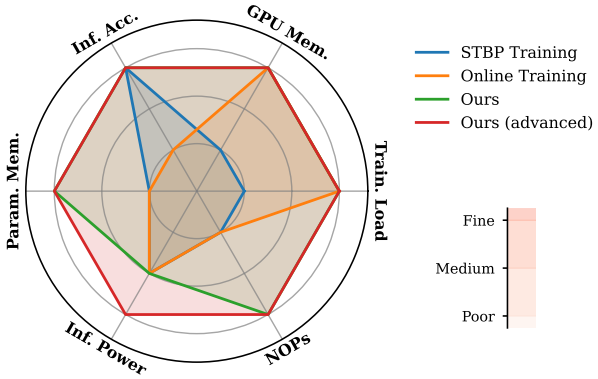


Figure 1. In comparison of STBP and vanilla online learning paradigms, our schemes achieve comprehensive optimization towards multiple performance metrics of SNN training (Train. Load, GPU Mem.) and inference (Inf. Acc., Param. Mem., Inf. Power, NOPs) stages.

Spatial-Temporal Back-propagation (STBP) is one of the most significant training algorithm in the supervised learning domain of SNNs currently [42]. By introducing the concepts of temporal dimension and surrogate gradient, STBP simultaneously tackles the Markov property and non-

*Corresponding author.

differentiable problem of SNNs existed in the forward propagation and firing process. However, although STBP training has significantly improved the learning performance and generalization ability of SNNs [34, 36, 41], as its back-propagation chain has the property of temporal dependency, the GPU memory will inevitably boost linearly with the number of time-steps. This phenomenon greatly increases the training burden and hinders the further application of SNNs to complex scenarios [23, 47] and advanced spiking models [14, 19].

To address this problem, researchers have transferred the idea of online learning to the STBP training framework [30, 44], which means that by detaching the temporal dependent gradient terms, SNNs can immediately perform back-propagation at any time-step. This scheme ensures that the corresponding GPU memory is independent of the training time-steps and remains constant, effectively alleviating the problem of computation memory explosion. However, as shown in Fig.1, the current proposed methods based on online learning still have the following defects: (i) the degradation of inference accuracy caused by the discrepancy between forward and backward propagation, (ii) inference deployment without advantages.

The reason for the first defect is that the surrogate function of spiking neurons is generally related to the value of membrane potential and the spike sequence is usually unevenly distributed in the temporal dimension, making the temporal dependent gradients different from each other and unable to merge with the back-propagation chain along the spatial dimension. In this case, when online learning frameworks detach temporal dependent gradients, it will lead to inconsistency between forward and backward propagation, while the gradient misalignment inherent in the surrogate function will further intensify this phenomenon, resulting in learning performance degradation. The second defect refers to the fact that current online learning methods merely focus on optimizing training GPU memory. However, the obtained SNN does not have additional advantages on various performance metrics compared to the corresponding model trained by STBP in the inference stage, which seriously damages the practical application value of online learning.

Based on the above discussion, we propose the concept of Hybrid-Driven Leaky Integrate-and-Fire (HD-LIF) model family for efficient online learning. On the one hand, HD-LIF model can effectively separate backward gradients from the temporal direction and integrate them into the spatial dimension; on the other hand, it can be efficiently combined with various techniques for optimizing SNN inference overhead, achieving plug-and-play online training and deployment. Our contributions are summarized as follow:

- Compared to vanilla spiking models, we point out that HD-LIF model has more superior gradient separability and alignment from the theoretical perspective, which is conducive to achieving high-performance online training.
- The model family covers multiple variant models adapted to different scenarios. Among them, Parallel HD-LIF can significantly reduce Neural Operations (NOPs) in the inference stage; Mem-BN HD-LIF, which is the version based on membrane potential batch-normalization, can effectively regulate the degree of gradient separability and be re-parameterized into vanilla HD-LIF model, as well as combining with efficient attention mechanism.
- For various datasets with different data-scale and data-type, our learning framework achieves state-of-the-art (SoTA) performance in multiple dimensions. For example, we reach top-1 accuracy of 78.61% on CIFAR-100 dataset under the condition of saving $10\times$ parameter memory, $11\times$ inference power and 30% NOPs.

2. Related Works

Recurrent learning algorithms for SNNs. Considering the similarity in computational mechanisms between SNNs and Recurrent Neural Networks (RNNs), Wu *et al*. [42] and Neftci *et al*. [32] transferred the Back-propagation Through Time (BPTT) method from RNNs to the supervised learning field of SNNs and utilized surrogate functions to tackle the non-differentiable problem existed in the spike firing process, which is called the STBP training algorithm. On this basis, Li *et al*. [27], Guo *et al*. [11] and Wang *et al*. [41] respectively attempted to start from the perspective of regulating the distribution about the backward gradient and membrane potential, introducing progressive surrogate functions and penalty terms. Tang *et al*. [38] proposed Relaxation LIF to mitigate gradient mismatch in deep SNN training. Deng *et al*. [4] proposed a target learning function which comprehensively considers the SNN output distribution within each time-step, which is particularly suitable for neuromorphic sequential data. To further improve the learning stability and performance of SNNs, various BatchNorm (BN) modules [6, 12, 21, 52] and attention mechanisms [34, 46, 47, 53] have been proposed successively, which capture the representation information contained in spike sequences from multiple dimensions, including spatial-wise, temporal-wise and channel-wise. Recently, advanced spiking models have become a focus of academic attention. Researchers have proposed a variety of neuron models with stronger dynamic properties and memorizing capabilities around membrane-related parameters [7, 48], firing mechanism [13, 35] and dendrite structure [14, 39], promoting deeper exploration towards brain-inspired intelligence. LIAF-Net [43] adopt a similar model structure to this work and combine it with a series of RNN-related modules, but they aim to achieve lightweight information processing rather than efficient online training and deployment. In addition, a spatial-temporal back-propagation algorithm based on spike firing time [1, 22, 51, 54] has also attracted

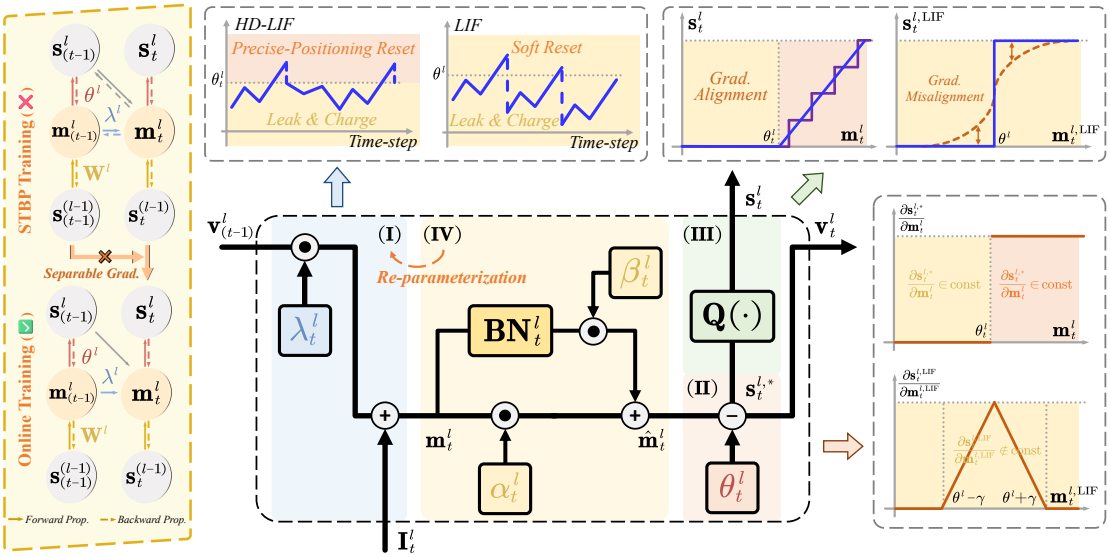


Figure 2. An overview of HD-LIF model. Compared to vanilla LIF, HD-LIF (middle-bottom) can enhance the gradient separability (right-bottom) and alignment (right-top) through the hybrid-driven spiking calculation mechanism (middle-top), thereby overcoming the discrepancy existed between the forward and backward process of conventional online training (left).

widespread attention. However, this series of methods are currently limited by high computational complexity and unstable training process, which cannot be effectively applied to complex network backbones and learning scenarios.

Online learning algorithm for SNNs. Although STBP learning algorithm promotes SNNs to join the club of high-performance models, it also brings severe computational burden to SNNs during the training phase, especially the GPU memory that will increase linearly with the number of time-steps. Xiao *et al*. [44] transferred the idea of online learning to the domain of SNN direct training, which splits the back-propagation chain by ignoring the backward gradients with temporal dependencies, making the training GPU memory independent of time-steps. On this basis, Meng *et al*. [30] proposed a selective back-propagation scheme based on online learning, which significantly improves training efficiency. Yang *et al*. [45] combined online learning with ANN-SNN knowledge distillation, further accelerating the training convergence speed of SNNs. Zhu *et al*. [55] proposed a brand-new BN module suitable for online learning, which enhances the stability of gradient calculation by considering the global mean and standard deviation in the temporal dimension. To enrich the neurodynamic property of online learning, Jiang *et al*. [20] introduced the difference of membrane potential between adjacent time-steps as a feature term into the backward gradient calculation. Inspired by the architecture of reversed network, Zhang *et al*. [50] and Hu *et al*. [17] respectively proposed reversible memory-efficient training algorithms from spatial and temporal perspectives. This type of algorithm can ensure computational consistency between online and STBP learning under the condition of occupying constant GPU memory, but it requires bi-directional computation towards all intermediate variables, which inevitably increases computational overhead. Alternatively, FPTT with liquid time-constant neurons [49] enables accurate online training of long sequences with constant memory.

3. Preliminaries

Leaky Integrate-and-Fire (LIF) model. The current mainstream spiking model used in SNN community is LIF model [9], which involves three core calculation processes, *i.e*. charging, firing and resetting. As shown in Eq.(1), at each time-step, LIF model will receive the input current $\mathbf{I}_t^{l,\mathrm{LIF}}$ and refer to the previous residual potential $\mathbf{v}_{(t-1)}^{l,\mathrm{LIF}}$, then accumulate the corresponding membrane potential $\mathbf{m}_t^{l,\mathrm{LIF}}$. When $\mathbf{m}_t^{l,\mathrm{LIF}}$ has exceeded the firing threshold θ^l, a spike $\mathbf{s}_t^{l,\mathrm{LIF}}$ will be transmitted to the post-synaptic layer and $\mathbf{m}_t^{l,\mathrm{LIF}}$ will be reset. Here $\mathbf{W}^l$ denotes the synaptic weight and λ^l represents the membrane leakage parameter.

$$\mathbf{m}_t^{l,\mathrm{LIF}} = \lambda^l \odot \mathbf{v}_{(t-1)}^{l,\mathrm{LIF}} + \mathbf{I}_t^{l,\mathrm{LIF}},\ \mathbf{v}_t^{l,\mathrm{LIF}} = \mathbf{m}_t^{l,\mathrm{LIF}} - \mathbf{s}_t^{l,\mathrm{LIF}},$$
$$\mathbf{I}_t^{l,\mathrm{LIF}} = \mathbf{W}^l \mathbf{s}_t^{(l-1),\mathrm{LIF}},\ \mathbf{s}_t^{l,\mathrm{LIF}} = \begin{cases} \theta^l, & \mathbf{m}_t^{l,\mathrm{LIF}} \geq \theta^l \\ 0, & \text{otherwise} \end{cases}. \quad (1)$$

STBP Training. To effectively train LIF model, the back-propagation procedure of SNNs usually chooses to expand along both spatial and temporal dimensions. We use $\mathcal{L}$ to denote the target loss function. As shown in Eq.(3), $\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}$

will depend on both $\frac{\partial \mathcal{L}}{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{m}_{(t+1)}^{l,\mathrm{LIF}}}$ simultaneously, while the non-differentiable problem of $\frac{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}$ is tackled through calculating approximate surrogate functions. Although STBP training enables SNN to achieve relatively superior performance, it inevitably causes severe GPU memory overhead during the training process, which will increase linearly with the number of time-steps.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}\frac{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}}_{\textit{spatial dimension}} + \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{m}_{(t+1)}^{l,\mathrm{LIF}}}\frac{\partial \mathbf{m}_{(t+1)}^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}}_{\textit{temporal dimension}}. \quad (2)$$

$$\nabla_{\mathbf{W}^l}\mathcal{L} = \sum_{t=1}^{T}\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}\frac{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}{\partial \mathbf{W}^l},$$

$$\frac{\partial \mathbf{m}_{(t+1)}^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}} = \lambda^l + \frac{\partial \mathbf{m}_{(t+1)}^{l,\mathrm{LIF}}}{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}\frac{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}. \quad (3)$$

Online Training. To avoid the issue of training memory overhead, a feasible solution is to detach $\frac{\partial \mathcal{L}}{\partial \mathbf{m}_{(t+1)}^{l,\mathrm{LIF}}}\frac{\partial \mathbf{m}_{(t+1)}^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}$ from the gradient calculation graph, thereby making the back-propagation chain independent in the temporal dimension. Online training enables SNNs to update gradients at any time-step, keeping the GPU memory at a constant level.

4. Methods

4.1. Overcoming the back-propagation discrepancy of online training

Eq.(3) unfolds the STBP back-propagation chain recursively from back to front along the temporal dimension, but one can find that the backward gradients can also be rewritten as a cumulative form: $\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}\frac{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}} + \sum_{i=t+1}^{T}\frac{\partial \mathcal{L}}{\partial \mathbf{s}_i^{l,\mathrm{LIF}}}\frac{\partial \mathbf{s}_i^{l,\mathrm{LIF}}}{\partial \mathbf{m}_i^{l,\mathrm{LIF}}}\prod_{j=t+1}^{i}\frac{\partial \mathbf{m}_j^{l,\mathrm{LIF}}}{\partial \mathbf{m}_{(j-1)}^{l,\mathrm{LIF}}}$. On this basis, we first point out the concept of Separable Backward Gradient:

Definition 4.1. Assume $\frac{\partial \mathcal{L}}{\partial \mathbf{s}_1^{l,\mathrm{LIF}}} = \ldots = \frac{\partial \mathcal{L}}{\partial \mathbf{s}_T^{l,\mathrm{LIF}}}$, we can rewrite $\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}\sum_{i=t}^{T}\boldsymbol{\epsilon}^l[i,t]$, here $\boldsymbol{\epsilon}^l[i,t]$ denotes the temporal gradient contribution weight of the i-th step w.r.t. the t-th step:

$$\boldsymbol{\epsilon}^l[i,t] \in \left\{\frac{\partial \mathbf{s}_i^{l,\mathrm{LIF}}}{\partial \mathbf{m}_i^{l,\mathrm{LIF}}}\prod_{j=t+1}^{i}\frac{\partial \mathbf{m}_j^{l,\mathrm{LIF}}}{\partial \mathbf{m}_{(j-1)}^{l,\mathrm{LIF}}},\ i>t;\ \frac{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}},\ i=t\right\}.$$

If $\forall t, i \in [1,T]$, $\boldsymbol{\epsilon}^l[i,t]$ is a constant value, then we can further have $\left(\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}\right)_{Online} \Leftrightarrow \left(\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}}\right)_{STBP}$. Here $\Leftrightarrow$ denotes the equivalence of gradient calculation. The gradient at this point is called Separable Backward Gradient.

When the precondition of Definition 4.1 holds true, the back-propagation chain can be considered separable in the temporal dimension and the backward gradient of online training can be seamlessly transformed from that of STBP training. Unfortunately, as shown in Fig.2, vanilla STBP training generally requires surrogate gradient functions which are related to the membrane potential value (*e.g.* Triangle Function: $\frac{\partial \mathbf{s}_t^{l,\mathrm{LIF}}}{\partial \mathbf{m}_t^{l,\mathrm{LIF}}} = \frac{1}{\gamma^2}\max\left(\gamma - |\mathbf{m}_t^{l,\mathrm{LIF}} - \theta^l|, 0\right)$), causing $\boldsymbol{\epsilon}^l[i,t] = \mathcal{F}(\mathbf{m}_t^{l,\mathrm{LIF}}, ..., \mathbf{m}_i^{l,\mathrm{LIF}})$. Considering that the specific values of membrane potential along temporal dimension are unpredictable and irregular, $\boldsymbol{\epsilon}^l[i,t]$ will not satisfy the condition of constant value. Therefore, current online training cannot effectively overcome the discrepancy between forward and backward propagation, which limits its learning precision.

To tackle this problem, we first propose vanilla HD-LIF, which is an advanced spiking model suitable for online training. In Eq.(4) and Fig.2, compared to vanilla LIF model, HD-LIF model retains the inherent membrane potential calculation mechanism of spiking models within the region below θ^l, while utilizing a new reset mechanism called Precise-Positioning Reset (P2-Reset) in the region above θ^l. Specifically, after each firing process, HD-LIF model will reset the value of $\mathbf{v}^l$ to θ^l and transmit spikes corresponding to the reset strength:

$$\begin{aligned}
&\mathbf{m}_t^l = \lambda_t^l \odot \mathbf{v}_{(t-1)}^l + \mathbf{I}_t^l,\ \mathbf{v}_t^l = \mathbf{m}_t^l - \mathbf{s}_t^{l,*},\\
&\mathbf{I}_t^l = \hat{\mathbf{W}}^l\mathbf{s}_t^{(l-1)},\ \mathbf{s}_t^{l,*} = \begin{cases}\mathbf{m}_t^l - \theta_t^l, & \mathbf{m}_t^l \geq \theta_t^l\\ 0, & \text{otherwise}\end{cases},\\
&\mathbf{s}_t^l = \mathbf{Q}(\mathbf{s}_t^{l,*}, s, n, \tau) = s\cdot\mathrm{clip}\left(\left\lfloor\frac{\mathbf{s}_t^{l,*}}{s}\right\rfloor_\tau, 0, 2^n - 1 + \tau\right).
\end{aligned} \quad (4)$$

Here $\hat{\mathbf{W}}^l$ adopts the learning modes of 1-bit [28] or 1.5-bit [25], which compresses the original weight to a ultra-low number of bits. Specifically, the 1-bit scheme ($\{-1,+1\}$) aims to obtain the optimal compression for synaptic parameters, while the 1.5-bit scheme ($\{0,\pm 1\}$) will further reduce the number of Synaptic Operations (SOPs) and power consumption by promoting the sparsity in synaptic weights. For $\lfloor x\rceil_\tau$, we define $\lfloor x\rceil_\tau = \lfloor x\rceil, \tau = 0; \lfloor x\rceil_\tau = x, \tau \to +\infty$. n and s denote the compressed bit-width and scaling factor. According to Eq.(4), we can further derive the back-propagation chain of HD-LIF model during the online training process:

$$\nabla_{\mathbf{W}^l}\mathcal{L} = \sum_{t=1}^{T}\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^l}\frac{\partial \mathbf{m}_t^l}{\partial \hat{\mathbf{W}}^l}\frac{\partial \hat{\mathbf{W}}^l}{\partial \mathbf{W}^l},\ \frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^l} = \frac{\partial \mathcal{L}}{\partial \mathbf{s}_t^l}\frac{\partial \mathbf{s}_t^l}{\partial \mathbf{s}_t^{l,*}}\frac{\partial \mathbf{s}_t^{l,*}}{\partial \mathbf{m}_t^l}. \quad (5)$$

For HD-LIF model, on the one hand, $\frac{\partial \mathbf{s}_t^{l,*}}{\partial \mathbf{m}_t^l}$ remains constant in $(-\infty, \theta_t^l]$ and $(\theta_t^l, +\infty)$ respectively, which makes the surrogate gradient of HD-LIF model independent of the

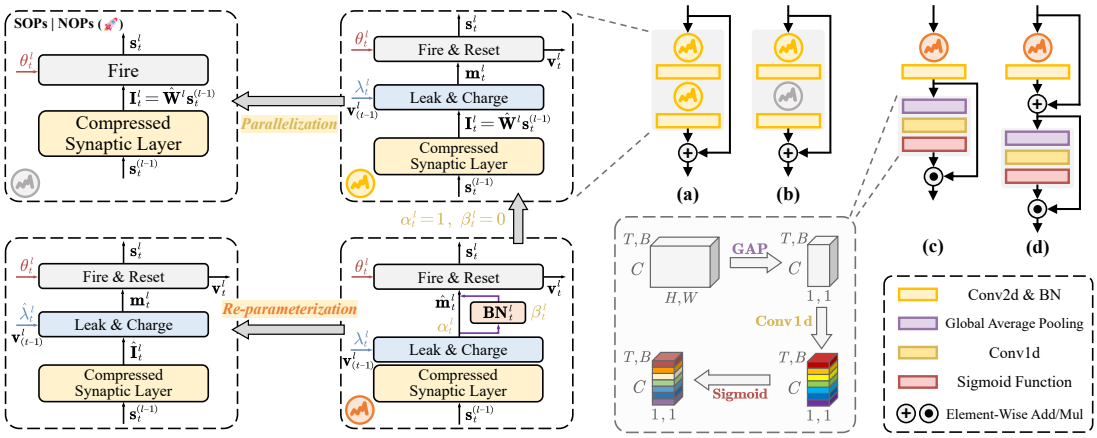


Figure 3. The structural description for SNN blocks based on HD-LIF model family. (a): vanilla version, (b): parallel version (w/ Parallel HD-LIF), (c)-(d): SECA_I and SECA_II (w/ Mem-BN HD-LIF).

corresponding membrane potential value, improving gradient separability along the temporal dimension; on the other hand, there is no non-differentiable problem between $\mathbf{s}_t^l$ and $\mathbf{m}_t^l$ during the firing process, ensuring gradient alignment along the spatial dimension. Based on the above observations, we can further propose the following theorem:

Theorem 4.2. *For HD-LIF model under the condition of online training, combining with Definition 4.1, we will have:* $\forall t, i \in [1, T], t < i, \boldsymbol{\epsilon}^l[i,t] = \boldsymbol{\chi}^l[i,i] \prod_{j=t+1}^{i} \boldsymbol{\chi}^l[j, j-1]$, *here we define* $\boldsymbol{\chi}^l[i,i] = \frac{\partial \mathbf{s}_i^l}{\partial \mathbf{m}_i^l} \in \{0,1\}, \boldsymbol{\chi}^l[j,j-1] = \frac{\partial \mathbf{m}_j^l}{\partial \mathbf{m}_{j-1}^l} \in \{0, \lambda_j^l\}$. *(i) If* $\exists k \in [t, T], \boldsymbol{\chi}^l[k,k] = 1$, *then we can directly derive the gradient relationship between STBP and online training:* $\left(\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^l}\right)_{Online} = \frac{\boldsymbol{\chi}^l[t,t]}{\left(\boldsymbol{\chi}^l[t,t] + \sum_{i=t+1}^{T} \boldsymbol{\chi}^l[i,i] \prod_{j=t+1}^{i} \boldsymbol{\chi}^l[j,j-1]\right)} \left(\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^l}\right)_{STBP}$. *(ii) Under more general conditions,* $\left(\frac{\partial \mathcal{L}}{\partial \mathbf{m}_t^l}\right)_{STBP} \in \left\{1, \prod_{j=t+1}^{t^*} \lambda_j^l\right\} \cdot \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}_t^l}\right)$, *here* $\boldsymbol{\chi}^l[t^*, t^*] = 1 \wedge \forall k \in [t, t^*), \boldsymbol{\chi}^l[k,k] = 0$.

From Theorem 4.2, one can find that HD-LIF model can convert $\boldsymbol{\epsilon}^l[i,t]$ into a constant value belonging to a finite set, thereby achieving the separability of the backward gradients. On this basis, we set λ_t^l and θ_t^l as learnable membrane-related parameters for HD-LIF model at each time-step, enabling HD-LIF model to more adaptively regulate its learning gradients during online training.

4.2. HD-LIF model family

Eq.(4) describes the dynamic process of vanilla HD-LIF model, consisting of the regions shown in Fig.2(I)-(III). Although vanilla model is already enable to engage in online learning, we continue to explore more possible dynamic calculation schemes within the model based on this foundation, aiming to further enhance the computational efficiency and learning capability of HD-LIF model. We will combine the following variant versions of HD-LIF model with the vanilla version to form the HD-LIF model family, as illustrated in Fig.3.

Parallel HD-LIF model. Due to the fact that vanilla HD-LIF model is based on traditional serial computing processes, there will still be T MUL (for leakage process) and $2T$ ADD (for charging and resetting process) operations within the model during the inference stage. In other words, compared to the LIF model, vanilla HD-LIF model has no advantage in the inference overhead of neuron layers. Therefore, as shown in Fig.3(a)-(b), we can consider introducing a parallel version of the HD-LIF model at an appropriate ratio, *i.e.*, setting $\mathbf{s}_t^{l,*} := (\mathbf{I}_t^l \geq \theta_t^l)$. At this point, there is no leakage or charging process for neuron layers, NOPs are only composed of T ADD operations.

Mem-BN HD-LIF model. Conventional BN layers in SNNs are usually placed behind the corresponding synaptic convolutional layers along the spatial dimension to monitor the distribution of input current $[\mathbf{I}_1^l, ..., \mathbf{I}_T^l]$, ensuring the stability of STBP training and seamlessly integrating with the convolutional layers during the inference stage. However, for online training, considering the lack of temporal gradient terms, in addition to controlling the distribution of input current, we also need to pay attention to the distribution stability of the cumulative membrane potential $[\mathbf{m}_1^l, ..., \mathbf{m}_T^l]$ over time-steps. Thanks to the fusible property of the BN layer, we consider a batch-normalization scheme oriented towards membrane potential along the temporal dimension, which can exquisitely fuse BN layers with membrane-related parameters in the SNN inference stage, further enhancing the learning ability of HD-

LIF model without introducing additional computational costs, as shown in Fig.2(IV). Specifically, we introduce the following calculation process for Mem-BN in addition to Eq.(4):

$$\hat{\mathbf{m}}_t^l = \alpha_t^l \odot \mathbf{m}_t^l + \beta_t^l \odot \mathbf{BN}_t^l(\mathbf{m}_t^l),\ \mathbf{v}_t^l = \hat{\mathbf{m}}_t^l - \mathbf{s}_t^{l,*},$$
$$\mathbf{BN}_t^l(\mathbf{m}_t^l) = \gamma^l \cdot \frac{\mathbf{m}_t^l - \mu_t^l}{\sqrt{{\sigma_t^l}^2 + \epsilon}} + b^l. \tag{6}$$

Here μ_t^l and σ_t^l are the recorded mean and standard deviation of $\mathbf{m}_t^l$, while γ_t^l and ϵ, b_t^l denote the scaling and shifting factors of the BN layer. α_t^l and β_t^l are learnable parameters that regulate the normalization degree of $\mathbf{m}_t^l$. When $\alpha_t^l = 1, \beta_t^l = 0$, the Mem-BN version will degrade to the vanilla version mentioned in §4.1, which ensures its performance lower-bound.

For SNN inference, Mem-BN HD-LIF can be equivalently converted into vanilla HD-LIF model through the following re-parameterization process. Specifically, Eq.(6) can be first integrated into the following equation:

$$\hat{\mathbf{m}}_t^l = (\alpha_t^l + \frac{\gamma_t^l \cdot \beta_t^l}{\sqrt{{\sigma_t^l}^2 + \epsilon}})\mathbf{m}_t^l - \beta_t^l(\frac{\gamma_t^l \cdot \mu_t^l}{\sqrt{{\sigma_t^l}^2 + \epsilon}} - b_t^l). \tag{7}$$

Subsequently, we can merge the scaling and shifting terms w.r.t. $\mathbf{m}_t^l$ in Eq.(7) into membrane-related parameters at different positions, including membrane leakage parameters $[\lambda_1^l, ..., \lambda_T^l]$ and input current $[\mathbf{I}_1^l, ..., \mathbf{I}_t^l]$, so that Mem-BN HD-LIF still only involves the processes of leakage, charging and resetting during inference calculations. Here we set $\alpha_t^{l,*} = \alpha_t^l + \frac{\gamma_t^l \cdot \beta_t^l}{\sqrt{{\sigma_t^l}^2 + \epsilon}}, \beta_t^{l,*} = \beta_t^l(\frac{\gamma_t^l \cdot \mu_t^l}{\sqrt{{\sigma_t^l}^2 + \epsilon}} - b_t^l)$ and derive the final membrane-related parameters $\forall t, \hat{\lambda}_t^l = \alpha_t^{l,*}\lambda_t^l, \hat{\mathbf{I}}_t^l = \alpha_t^{l,*}\mathbf{I}_t^l - \beta_t^{l,*}$ after re-parameterization:

$$\begin{aligned}(\hat{\mathbf{m}}_t^l)_{\text{inference}} &= \alpha_t^{l,*}\mathbf{m}_t^l - \beta_t^{l,*} \\ &= \alpha_t^{l,*}(\lambda_t^l \odot \mathbf{v}_{(t-1)}^l + \mathbf{I}_t^l) - \beta_t^{l,*} \\ &= (\alpha_t^{l,*}\lambda_t^l) \odot \mathbf{v}_{(t-1)}^l + (\alpha_t^{l,*}\mathbf{I}_t^l - \beta_t^{l,*}) \\ &= \hat{\lambda}_t^l \odot \mathbf{v}_{(t-1)}^l + \hat{\mathbf{I}}_t^l \end{aligned} \tag{8}$$

4.3. Enhancing HD-LIF through efficient attention mechanism

Channel Attention mechanism [10, 16, 40] is usually inserted after the convolutional layers to further optimize the network performance. Considering that it is not suitable to introduce computation- or parameter-intensive attention mechanisms during online training and deployment, we transfer the idea of ECA [40] to the HD-LIF online training framework, then propose Spiking Efficient Channel Attention (SECA) mechanism, as shown in Eq.(9).

$$\text{SECA}(\mathbf{I}_t^l) = \sigma\big(\underbrace{\text{Conv1d}}_{\in \mathbb{R}^{1\times1\times K}}\big(\underbrace{\text{GAP}(\mathbf{I}_t^l)}_{\in \mathbb{R}^{B\times1\times C}}\big)\big) \odot \mathbf{I}_t^l. \tag{9}$$

Here the input current $\mathbf{I}_t^l \in \mathbb{R}^{B\times C\times H\times W}$ will be compressed to $\mathbb{R}^{B\times C\times1\times1}$ through the Global Average Pooling (GAP) layer, then Conv1d$(\cdot)$ and Sigmoid Function $\sigma(\cdot)$ will be used to capture and activate the attention scores among different channels, ultimately merging with the shortcut path. Considering that the HD-LIF model can convey enough information representation at each time-step, we enable the spike sequence to share the weight of SECA in the temporal dimension. Due to its ultra-low parameter quantity and computational load (1 Conv1d layer with $O(K)$ parameters and $O(KC)$ operations), SECA can further enhance the learning ability of HD-LIF SNNs under the condition of hardly affecting its online training and deployment. In addition, as shown in Fig.3(c)-(d) and Eq.(10), we further propose two variants for SECA:

$$\text{SECA}_{\text{I}}^l(\cdot) : \text{SECA}(\mathbf{I}_t^l),\ \text{SECA}_{\text{II}}^l(\cdot) : \text{SECA}(\mathbf{I}_t^{(l-1)} + \mathbf{I}_t^l). \tag{10}$$

Here $\text{SECA}_{\text{I}}^l(\cdot)$ is the vanilla channel attention mechanism, while considering the shortcomings of compressed synaptic layers in feature extraction, $\text{SECA}_{\text{II}}^l(\cdot)$ combines the input currents from both pre-synaptic and post-synaptic layers to further enhance the effectiveness of the attention mechanism. To achieve efficient online training for HD-LIF models, we draw inspiration from the scheme proposed in [30], where for each training batch, we merely perform gradient update for synaptic weights and corresponding membrane-related parameters at a randomly selected time-step. The overall procedure has been summarized in Algorithm 1.

5. Experiments

To validate the superiority of the HD-LIF online learning framework, we evaluate its performance across a diverse set of datasets spanning different scales and types, including CIFAR-10/100 [24], ImageNet-200/1K [3] and DVS-CIFAR10 [26]. We compare against representative prior works in the domain of SNN mainstream and online training on ResNet [15, 18] and VGG [37] backbones. Specifically, all ResNet cases on static datasets use the block structure of Fig.3(b) by default; when adopting neuromorphic data and SECA, we replace vanilla HD-LIF in the block with Mem-BN HD-LIF to further enhance the learning ability of SNNs. The SECA module based on ResNet chooses the structure mentioned in Fig.3(d). For VGG structure, we utilize the vanilla HD-LIF model for static datasets and Mem-BN version for neuromorphic datasets. More training and implementation details have been provided in Appendix.

5.1. Comparison with previous SoTA works

CIFAR-10 & CIFAR-100. As shown in Tab.1, compared to conventional STBP and online learning framework, our scheme utilizes advanced spiking calculation mechanism

Table 1. Comparison with previous SoTA works from SNN mainstream or online learning paradigms. Here Param.(MB) denotes the parameter memory of convolutional and linear layers in the corresponding standard network backbone.

Dataset	Method	Architecture	Param.(MB)	Type	Time-steps	Accuracy(%)
CIFAR-10	STBP-tdBN [52]	ResNet-19	50.48	STBP Training	4	92.92
	Dspike [27]	ResNet-18	44.66	STBP Training	4	93.66
	TET [4]	ResNet-19	50.48	STBP Training	4	94.44
	GLIF [48]	ResNet-18	44.66	STBP Training	4, 6	94.67, 94.88
	SLTT [30]	ResNet-18	44.66	Online Training	6	94.44
	Ours	**ResNet-18**	**2.82**	**Online Training**	**4**	**95.59**
CIFAR-100	Dspike [27]	ResNet-18	44.84	STBP Training	4	73.35
	TET [4]	ResNet-19	50.57	STBP Training	4	74.47
	GLIF [48]	ResNet-18	44.84	STBP Training	4, 6	76.42, 77.28
	SLTT [30]	ResNet-18	44.84	Online Training	6	74.38
	Ours	**ResNet-18**	**3.00**	**Online Training**	**4**	**78.45**
ImageNet-200	DCT [8]	VGG-13†	38.02*	Hybrid Training	125	56.90
	Offline-LTL [45]	VGG-13†	38.02*	STBP Training	16	55.37
	Online-LTL [45]			Online Training	16	54.82
	ASGL [41]	VGG-13	38.02	STBP Training	4, 8	56.57, 56.81
	Ours	**VGG-13**	**2.77**	**Online Training**	**4**	**60.68**
ImageNet-1k	STBP-tdBN [52]	ResNet-34	87.12	STBP Training	6	63.72
	TET [4]	ResNet-34	87.12	STBP Training	6	64.79
	OTTT [44]	ResNet-34	87.12	Online Training	6	65.15
	SLTT [30]	ResNet-34	87.12	Online Training	6	66.19
	Ours	**ResNet-34**	**10.06**	**Online Training**	**4**	**69.77**
DVS-CIFAR10	STBP-tdBN [52]	ResNet-19	50.48	STBP Training	10	67.80
	Dspike [27]	ResNet-18	44.66	STBP Training	10	75.40
	OTTT [44]	VGG-SNN	37.05	Online Training	10	76.30
	NDOT [20]	VGG-SNN	37.05	Online Training	10	77.50
	Ours	**ResNet-18**	**2.81**	**Online Training**	**10**	**81.70**
		VGG-SNN	**2.49**		**10**	**83.00**

† utilizes fully-connected layers with relatively large parameter quantity.

Table 2. Ablation studies for HD-LIF models with different configuration on CIFAR-100, ResNet-18.

Model Config.	T. Speed(s/epoch)	GPU Mem.(GB)	Param.(MB)	Acc.(%)	SOPs(M)	NOPs(M)	Energy(mJ)
LIF	20.52	1.50	44.84	71.75	273.02	6.59	0.25
HD-LIF	37.93	1.68	4.40	**80.16**	284.49	6.59	0.26
HD-LIF†	40.91	1.92	4.40	79.62	233.84	6.59	0.03
HD-LIF‡	**35.39**	**1.44**	4.40	78.82	254.08	4.62	0.23
HD-LIF†,‡	38.50	1.70	**4.40**	78.61	**190.13**	**4.62**	**0.02**

† denotes compressing the firing spikes to 4-bits, *i.e.* $\tau = 0, n = 4$.
‡ denotes substituting vanilla HD-LIF with Parallel HD-LIF at a proportion of 50%.

and synaptic weight compression technique, which saves approximately $15\times$ synaptic parameter memory without performance degradation. In addition, we achieve higher learning precision within the same or fewer time-steps. For example, our method outperforms GLIF [48] and SLTT [30] with accuracies of 2.03% and 4.07% respectively on CIFAR-100, ResNet-18.

ImageNet-200 & ImageNet-1k. For large-scale datasets, our HD-LIF model has also demonstrated significant advantages. For instance, we respectively achieve accuracies of 60.68% and 69.77% on ImageNet-200 and ImageNet-1k, which exceeds the corresponding online learning methods [30, 45] within fewer time-steps and saves more than 88% of synaptic parameter memory.

DVS-CIFAR10. Our method can also achieve effective information extraction for neuromorphic data. From Tab.2, one can note that our learning precision is 6.30% higher than Dspike [27] and 5.50% higher than NDOT [20] under the condition of utilizing the identical network backbone and time-steps.

5.2. Detailed analysis for performance metric

Table 2 shows the comparison of online learning frameworks based on LIF and HD-LIF model in multiple performance metrics of training and inference stages. For Speed and GPU memory during the training stage, since HD-LIF involves more learnable parameters, our approach incurs slightly higher overhead than vanilla LIF model, but the to-

Table 3. Parameter memory and accuracy of SNNs before and after utilizing SECA.

Method	CIFAR-10, ResNet-18		CIFAR-100, ResNet-18		DVS-CIFAR10, ResNet-18	
	Param.(MB)	Acc.(%)	Param.(MB)	Acc.(%)	Param.(MB)	Acc.(%)
HD-LIF	2.82	95.59	3.00	78.45	2.81	81.70
HD-LIF (Mem-BN & SECA)	**2.82**	**95.91** (+0.32)	**3.00**	**79.33** (+0.88)	**2.81**	**83.50** (+1.80)

Algorithm 1 Online learning framework for HD-LIF model family.

Input: SNN model $f_{\text{SNN}}(\mathbf{W}, \lambda, \theta, \alpha, \beta)$ with L layers; training dataset $\mathbf{D}$; time-steps T

Output: Deployable SNN model $f_{\text{SNN}}(\hat{\mathbf{W}}, \hat{\lambda}, \theta)$

```
# Online Training Stage
Define gradient update time for training batches: G = {t ∈ randint_{1:T} for x_data ∈ D}
for x_data, x_label ∈ D; t = 1 to T; l = 1 to L do
    perform forward propagation ∈ {s_t^l = Q(Ŵ^l s_t^{(l-1)} − θ_t^l, s, n, τ), Eq.(4), Eq.(6)}
    for SECA, calculate and merge the attention scores in Eqs.(9-10)
    if t is equal to G_{x_data} then
        perform back-propagation in Eq.(5), update Ŵ^l and learnable membrane-related parameters
    end if
end for
# Online Deployment Stage
for t = 1 to T; l = 1 to L do
    convert W^l to Ŵ^l
    if current neuron layer is Mem-BN HD-LIF then
        re-parameterize λ_t^l, α_t^l, β_t^l, BN_t^l(·) into λ̂_t^l, Î_t^l through Eq.(8)
    end if
end for
```

tal computational overhead still remains at a constant-level complexity. In the inference stage, the HD-LIF framework can significantly surpass vanilla online training in parameter memory and inference accuracy, combining with parallel calculation mode, it can further save 30% NOPs. In addition, it is worth noting that HD-LIF can maintain superior learning ability even when significantly compressing the information density of firing spikes, thereby achieving low-memory, low-power and high-precision online inference. For example, HD-LIF can outperform LIF by 7.87% while saving 10× parameter memory and 9× power consumption.

5.3. Performance validation for SECA

As shown in Tab.3, we explore the network performance before and after inserting SECA modules for multiple datasets. One can note that SECA hardly introduces additional parameter quantity, but can provide extra precision improvement for synaptic layers. Among them, after introducing SECA mechanism, the performance improvement of HD-LIF online learning on DVS-CIFAR10, ResNet-18 even reaches 1.80%, which effectively compensates for the potential accuracy loss caused by the compression of synaptic weights.

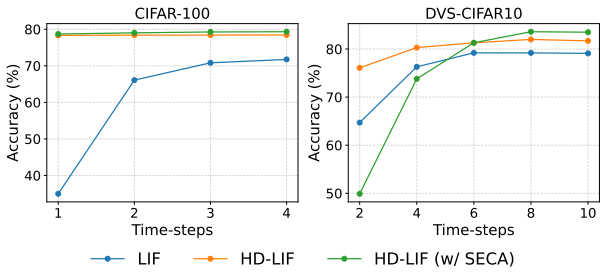


Figure 4. Accuracies of HD-LIF & LIF models over time-steps.

5.4. Temporal processing property of HD-LIF

As shown in Fig.4, due to the ability of HD-LIF to transmit richer spiking information at each time-step, for static datasets, HD-LIF models are enable to reach approximately SoTA performance at the first time-step, reflecting their similarity to ANN; for neuromorphic data, HD-LIF can gradually accumulate information along the temporal dimension, resulting in an increase in inference accuracy with the number of time-steps, which maintains the same property as vanilla spiking models. The above results demonstrate the duality of the hybrid-driven mechanism. Here Tab.2 and Fig.4 adopt unified LIF baselines.

6. Conclusions

In this paper, we systematically analyze the deficiencies of conventional online training, then propose a novel online learning framework based on the HD-LIF model family, which effectively tackles the gradient inseparability and misalignment problem. By flexibly combining various optimization techniques, HD-LIF model can further achieve comprehensive online learning and deployment towards multiple performance evaluation metrics. Experimental results have verified that our proposed scheme can break through the limitations of the previous learning paradigm and provide further inspiration for the future development of online learning.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (U24B20140, 62422601, 62506011), Beijing Municipal Science and Technology Program (Z241100004224004), Beijing Nova Program (20230484362, 20240484703).

References

[1] Sander M Bohte, Joost N Kok, and Han La Poutre. Error-backpropagation in temporally encoded networks of spiking neurons. *Neurocomputing*, 48(1-4):17–37, 2002. 2

[2] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018. 1

[3] Jia Deng, Richard Socher, Lijia Li, Kai Li, and Feifei Li. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 6

[4] Shikuang Deng, Yuhang Li, Shanghang Zhang, and Shi Gu. Temporal efficient training of spiking neural network via gradient re-weighting. *ICLR*, 2022. 2, 7

[5] Jianhao Ding, Zhaofei Yu, Jian K Liu, and Tiejun Huang. Neuromorphic computing paradigms enhance robustness through spiking neural networks. *Nature Communications*, 16(1):10175, 2025. 1

[6] Chaoteng Duan, Jianhao Ding, Shiyan Chen, Zhaofei Yu, and Tiejun Huang. Temporal effective batch normalization in spiking neural networks. In *NeurIPS*, 2022. 2

[7] Wei Fang, Zhaofei Yu, Yanqi Chen, Timothee Masquelier, Tiejun Huang, and Yonghong Tian. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *ICCV*, 2021. 2

[8] Isha Garg, Sayeed Shafayet Chowdhury, and Kaushik Roy. DCT-SNN: Using dct to distribute spatial information over time for low-latency spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021. 7

[9] Wulfram Gerstner and Werner M Kistler. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge university press, 2002. 3

[10] Nianhui Guo, Joseph Bethge, Christoph Meinel, and Haojin Yang. Join the high accuracy club on imagenet with a binary neural network ticket. *arXiv preprint arXiv:2211.12933*, 2022. 6

[11] Yufei Guo, Xinyi Tong, Yuanpei Chen, Liwen Zhang, Xiaode Liu, Zhe Ma, and Xuhui Huang. RecDis-SNN: Rectifying membrane potential distribution for directly training spiking neural networks. In *CVPR*, 2022. 2

[12] Yufei Guo, Yuhan Zhang, Yuanpei Chen, Weihang Peng, Xiaode Liu, Liwen Zhang, Xuhui Huang, and Zhe Ma. Membrane potential batch normalization for spiking neural networks. In *ICCV*, 2023. 2

[13] Yufei Guo, Yuanpei Chen, Xiaode Liu, Weihang Peng, Yuhan Zhang, Xuhui Huang, and Zhe Ma. Ternary spike: Learning ternary spikes for spiking neural networks. In *AAAI*, 2024. 2

[14] Zecheng Hao, Xinyu Shi, Zihan Huang, Tong Bu, Zhaofei Yu, and Tiejun Huang. A progressive training framework for spiking neural networks with learnable multi-hierarchical model. In *ICLR*, 2024. 2

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 6

[16] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *CVPR*, 2018. 6

[17] JiaKui Hu, Man Yao, Xuerui Qiu, Yuhong Chou, Yuxuan Cai, Ning Qiao, Yonghong Tian, Bo Xu, and Guoqi Li. High-performance temporal reversible spiking neural networks with $O(L)$ training memory and $O(1)$ inference cost. In *ICML*, 2024. 3

[18] Yifan Hu, Lei Deng, Yujie Wu, Man Yao, and Guoqi Li. Advancing spiking neural networks towards deep residual learning. *TNNLS*, 36(2):2353–2367, 2024. 6

[19] Yulong Huang, Xiaopeng Lin, Hongwei Ren, Haotian Fu, Yue Zhou, Zunchang Liu, Biao Pan, and Bojun Cheng. Clif: Complementary leaky integrate-and-fire neuron for spiking neural networks. In *ICML*, 2024. 2

[20] Haiyan Jiang, Giulia De Masi, Huan Xiong, and Bin Gu. NDOT: Neuronal dynamics-based online training for spiking neural networks. In *ICML*, 2024. 3, 7

[21] Haiyan Jiang, Vincent Zoonekynd, Giulia De Masi, Bin Gu, and Huan Xiong. Tab: Temporal accumulated batch normalization in spiking neural networks. In *ICLR*, 2024. 2

[22] Jinseok Kim, Kyungsu Kim, and Jae-Joon Kim. Unifying activation- and timing-based learning rules for spiking neural networks. In *NeurIPS*, 2020. 2

[23] Seijoon Kim, Seongsik Park, Byunggook Na, and Sungroh Yoon. Spiking-YOLO: spiking neural network for energy-efficient object detection. In *AAAI*, 2020. 2

[24] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 6

[25] Fengfu Li, Bin Liu, Xiaoxing Wang, Bo Zhang, and Junchi Yan. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016. 4

[26] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li, and Luping Shi. Cifar10-dvs: an event-stream dataset for object classification. *Frontiers in Neuroscience*, 11, 2017. 6

[27] Yuhang Li, Yufei Guo, Shanghang Zhang, Shikuang Deng, Yongqing Hai, and Shi Gu. Differentiable spike: Rethinking gradient-descent for training spiking neural networks. In *NeurIPS*, 2021. 2, 7

[28] Zechun Liu, Zhiqiang Shen, Shichao Li, Koen Helwegen, Dong Huang, and Kwang-Ting Cheng. How do adam and training strategies help bnns optimization. In *ICML*, 2021. 4

[29] Wolfgang Maass. Networks of spiking neurons: the third generation of neural network models. *Neural Networks*, 10 (9):1659–1671, 1997. 1

[30] Qingyan Meng, Mingqing Xiao, Shen Yan, Yisen Wang, Zhouchen Lin, and Zhiquan Luo. Towards memory and time-efficient backpropagation for training spiking neural networks. In *ICCV*, 2023. 2, 3, 6, 7

[31] Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014. 1

[32] Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36 (6):51–63, 2019. 2

[33] Jing Pei, Lei Deng, Sen Song, Mingguo Zhao, Youhui Zhang, Shuang Wu, Guanrui Wang, Zhe Zou, Zhenzhi Wu, Wei He, et al. Towards artificial general intelligence with hybrid tianjic chip architecture. *Nature*, 572(7767):106–111, 2019. 1

[34] Xuerui Qiu, Rui-Jie Zhu, Yuhong Chou, Zhaorui Wang, Liang-jian Deng, and Guoqi Li. Gated attention coding for training high-performance and efficient spiking neural networks. In *AAAI*, 2024. 2

[35] Guobin Shen, Dongcheng Zhao, Tenglong Li, Jindong Li, and Yi Zeng. Are conventional snns really efficient? a perspective from network quantization. In *CVPR*, 2024. 2

[36] Xinyu Shi, Zecheng Hao, and Zhaofei Yu. Spikingresformer: Bridging resnet and vision transformer in spiking neural networks. In *CVPR*, 2024. 2

[37] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 6

[38] Jianxiong Tang, Jian-Huang Lai, Wei-Shi Zheng, Lingxiao Yang, and Xiaohua Xie. Relaxation LIF: A gradient-based spiking neuron for direct training deep spiking neural networks. *Neurocomputing*, 501:499–513, 2022. 2

[39] Lihao Wang and Zhaofei Yu. Autaptic synaptic circuit enhances spatio-temporal predictive learning of spiking neural networks. In *ICML*, 2024. 2

[40] Qilong Wang, Banggu Wu, Pengfei Zhu, Peihua Li, Wangmeng Zuo, and Qinghua Hu. Eca-net: Efficient channel attention for deep convolutional neural networks. In *CVPR*, 2020. 6

[41] Ziming Wang, Runhao Jiang, Shuang Lian, Rui Yan, and Huajin Tang. Adaptive smoothing gradient learning for spiking neural networks. In *ICML*, 2023. 2, 7

[42] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, and Luping Shi. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, 12:331, 2018. 1, 2

[43] Zhenzhi Wu, Hehui Zhang, Yihan Lin, Guoqi Li, Meng Wang, and Ye Tang. LIAF-Net: Leaky integrate and analog fire network for lightweight and efficient spatiotemporal information processing. *TNNLS*, 33(11):6249–6262, 2021. 2

[44] Mingqing Xiao, Qingyan Meng, Zongpeng Zhang, Di He, and Zhouchen Lin. Online training through time for spiking neural networks. In *NeurIPS*, 2022. 2, 3, 7

[45] Qu Yang, Jibin Wu, Malu Zhang, Yansong Chua, Xinchao Wang, and Haizhou Li. Training spiking neural networks with local tandem learning. In *NeurIPS*, 2022. 3, 7

[46] Man Yao, Guangshe Zhao, Hengyu Zhang, Yifan Hu, Lei Deng, Yonghong Tian, Bo Xu, and Guoqi Li. Attention spiking neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(8):9393–9410, 2023. 2

[47] Man Yao, Jiakui Hu, Tianxiang Hu, Yifan Xu, Zhaokun Zhou, Yonghong Tian, Bo Xu, and Guoqi Li. Spike-driven transformer v2: Meta spiking neural network architecture inspiring the design of next-generation neuromorphic chips. In *ICLR*, 2024. 2

[48] Xingting Yao, Fanrong Li, Zitao Mo, and Jian Cheng. GLIF: A unified gated leaky integrate-and-fire neuron for spiking neural networks. In *NeurIPS*, 2022. 2, 7

[49] Bojian Yin, Federico Corradi, and Sander M Bohté. Accurate online training of dynamical spiking neural networks through forward propagation through time. *Nature Machine Intelligence*, 5(5):518–527, 2023. 3

[50] Hong Zhang and Yu Zhang. Memory-efficient reversible spiking neural networks. In *AAAI*, 2024. 3

[51] Wenrui Zhang and Peng Li. Temporal spike sequence learning via backpropagation for deep spiking neural networks. In *NeurIPS*, 2020. 2

[52] Hanle Zheng, Yujie Wu, Lei Deng, Yifan Hu, and Guoqi Li. Going deeper with directly-trained larger spiking neural networks. In *AAAI*, 2021. 2, 7

[53] Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, and Yuan Li. Spikformer: When spiking neural network meets transformer. In *ICLR*, 2023. 2

[54] Yaoyu Zhu, Wei Fang, Xiaodong Xie, Tiejun Huang, and Zhaofei Yu. Exploring loss functions for time-based training strategy in spiking neural networks. In *NeurIPS*, 2023. 2

[55] Yaoyu Zhu, Jianhao Ding, Tiejun Huang, Xiaodong Xie, and Zhaofei Yu. Online stabilization of spiking neural networks. In *ICLR*, 2024. 3